

Week-5

Q1: What are the key limitations of traditional MapReduce that make **Spark** a preferred choice for modern big data processing?

Q2: Explain how Spark uses **In-Memory Computing** to speed up iterative machine learning algorithms compared to disk-based systems.

Q3: Write a code snippet to remove all **duplicate rows** from a DataFrame based on a specific set of columns: `user_id` and `transaction_date`.

Q4: Given a DataFrame `df_sales`, write a query to **filter** for rows where the region is 'West' and then **group by** `product_category` to find the average `sale_amount`.

Q5: What is the difference between `.na.drop()` and `.na.fill()`? Provide a code example of filling null values in a `status` column with the string 'Unknown'.

Q6: Write a query to find the **total count** of records for each `city` in a DataFrame, but only for cities where the count is greater than 100.

Q7: How does the **immutability** of Spark DataFrames affect how you perform "data cleaning" steps like dropping columns or renaming them?

Q8: Write a Spark command to **filter** a dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is 'Premium'.

Q9: When **cleaning a dataset**, why is it often better to handle null values before performing mathematical aggregations like `sum()` or `avg()`?

Q10: Write the code to **revise** a column named `raw_timestamp` by casting it to a `TimestampType` and renaming it to `event_time`.

Q11: Explain the "Shuffle" process that occurs during a **grouping** operation. Why is it considered a wide transformation?

Q12: Write a code snippet that identifies and **removes rows** where the `email` column contains null values OR the `username` is an empty string.

Q13: How do you use the `.agg()` function to calculate multiple statistics at once, such as the min, max, and mean of the `price` column?

Q14: In the context of **cleaning a dataset**, what is the risk of using `inferSchema=true` when your source data contains messy or inconsistent date formats?

Q15: Write a final processing pipeline that:

1. Filters out **duplicates**.
2. Fills **null** prices with 0.
3. Groups by `store_id` to calculate total revenue.